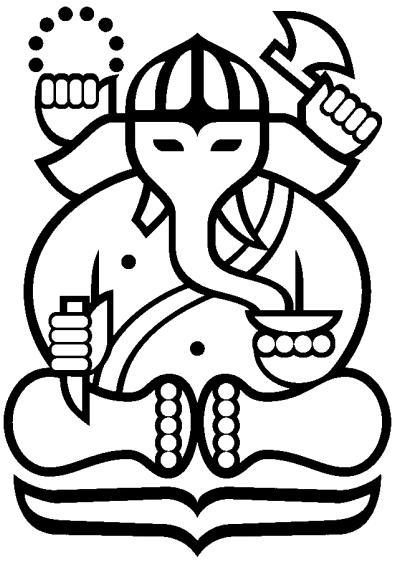
# LAPORAN PROJECT-2

## PEMODELAN DATA LANJUT

***Analitik tap-in/tap-out kartu transportasi umum menggunakan PostreSQL dan ClickHouse***

***Clickhouse sebagai DBMS NoSQL Columnar***




Disusun oleh:

Faqih Muhammad Syuhada 13523057

Program Studi Teknik informatika

Institut Teknologi Bandung


2026

# BAB I
# Deskripsi DBMS

**Deskripsi Clickhouse**

Pada tahun 2009 Alexey Milovidov mulai bereksperimen di Yandex tempat ia bekerja. “Apakah laporan analitik bisa dibuat realtime dari data mentah yang belum di agregasi?”. Eksperimen ini membuahkan hasil sehingga pada tahun 2012 masuk produksi sebagai mesin Yandex bernama Metrica sebuah web analitik. Pada 2016, Clickhouse dibuka sebagai *open source* dan pada tahun 2021 ClickHouse, Inc berdiir utnuk mengembangkan tenologi ini sehingga menjadi salah satu *cloud service.*

Clickhouse Umumnya digunakan untuk realtime analitik pada data yang terus bertambah dan jarang diubah berupa analitik web, produk, log, metrik. Perusahaan besar yang menggunakan Clickhouse antara lain Cloudeflare sebagai analitik trafik dengan 36 server, eBay untuk log, metrik dan event, Disney+ dengan penggunaan 395 TiB dan analitik.

Clickhouse dibuat bukan untuk OLTP sehingga dukungan transaksinya terbatas, tidak ada foreign key, update dan delete berupa tiruan yang aslinya menulis ulang data dan primary keynya tidak unik.

**Model Data**

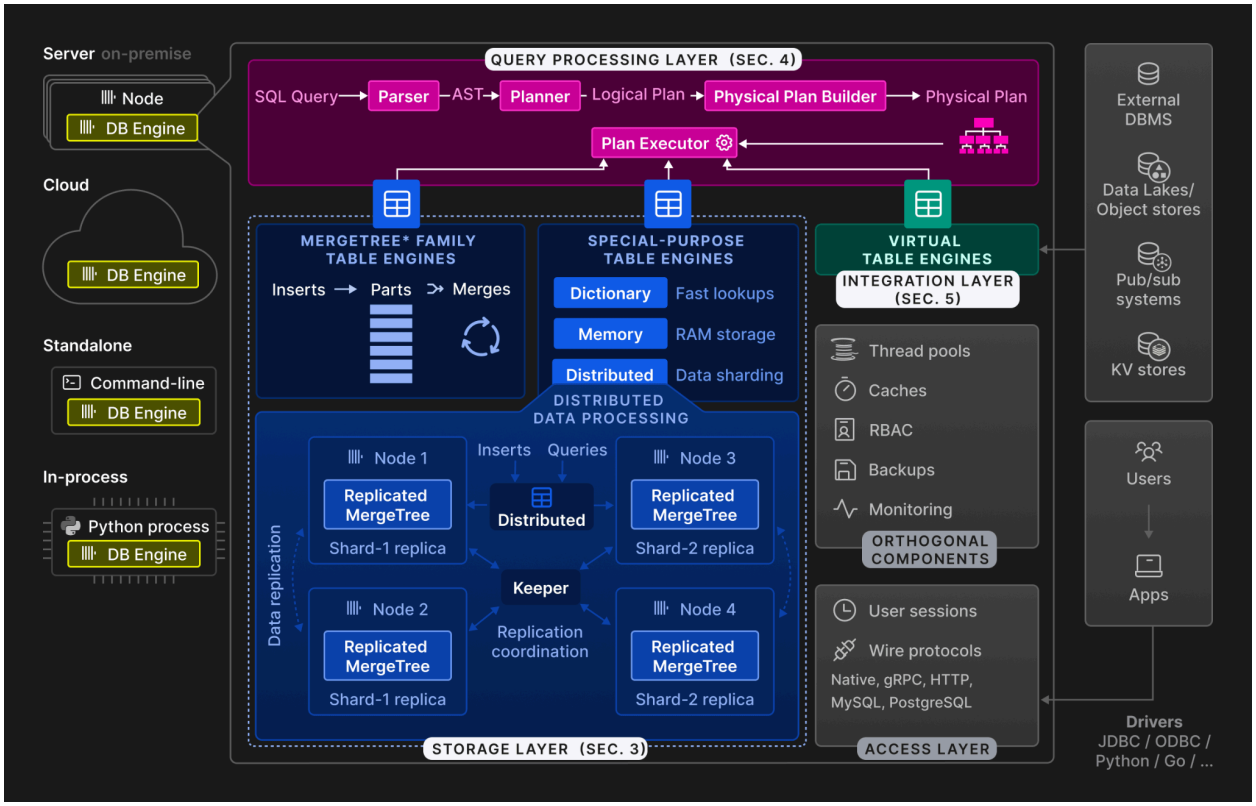
Model data dari Clickhouse disimpan sebagai tabel dengan schema heavy.  Dimana Clickhouse memiliki table engine yang berguna untuk menentukan cara data disimpan.

- Engine utamanya adalah Mergetree, dimana order by menentukan urutan secara fisik baris dan juga sparse primary index, Partition by memecah tabel jadi partisi agar rentang waktu bisa dilewati dan data lama gampang dihapus. Banyak varian dari Merge tree sendiri dimana ini menambah cara yang dilakukan saat merging atau replikasi.
- Tipe Data dari clickhouse sendiri banyak dan diguanakn untuk menentukan ukuran dari kolom yang ada.
- Index sekunder dimana diclickhouse digunakan sehingga bisa melewati blok yang berisi baris yang tidak ditunjuk.

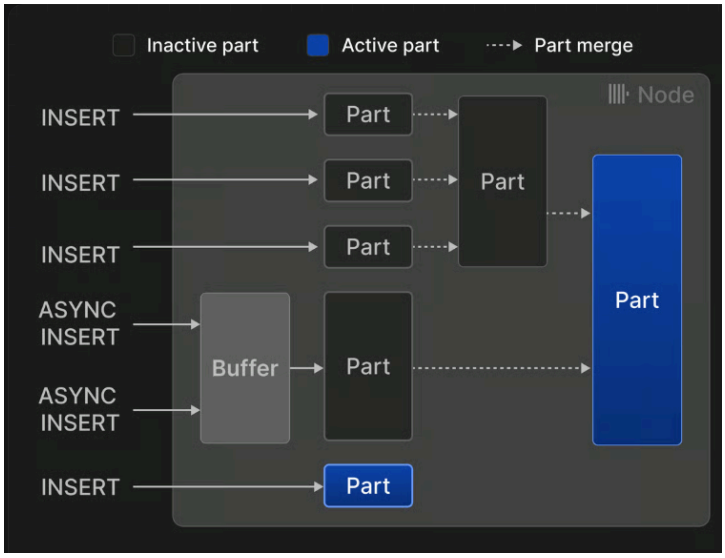
**Query**

Clickhouse memakai menggunakan bahasa SQL sebagai bahasa untuk query dan fungsi tambahan untuk analitik. Beberapa mekanisme yang ada di SQL berupa menulis data, merubah data merupakan update dan delete yang jalan sebagai tiruan. Kemampuan join juga didukung tapi tabelnya dibuat di memori sehingga penggabungan dimension dan fact sesuai dengan kebutuhan dilakukan sekali saja saat ETL.

**Arsitektur Sistem ClickHouse**

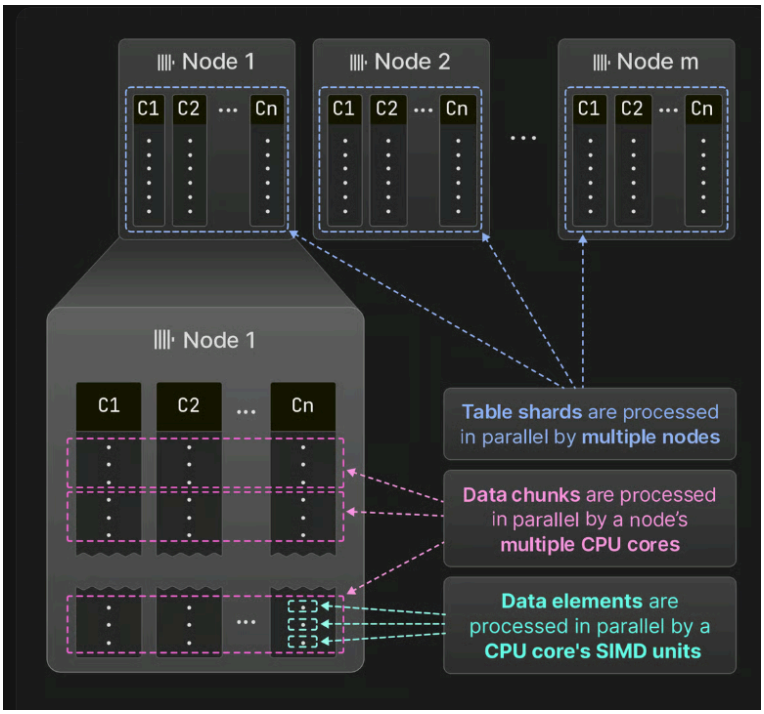



Pada arsitektur yang ada ClickHouse dibagi menjadi tiga layer, yaitu query processing, storage, dan integration layer, ditambah access layer yang mengatur user session dan protocol. Untuk analitik sendiri yang paling banyak menentukan kinerjanya adalah storage layer dan query processing layer.

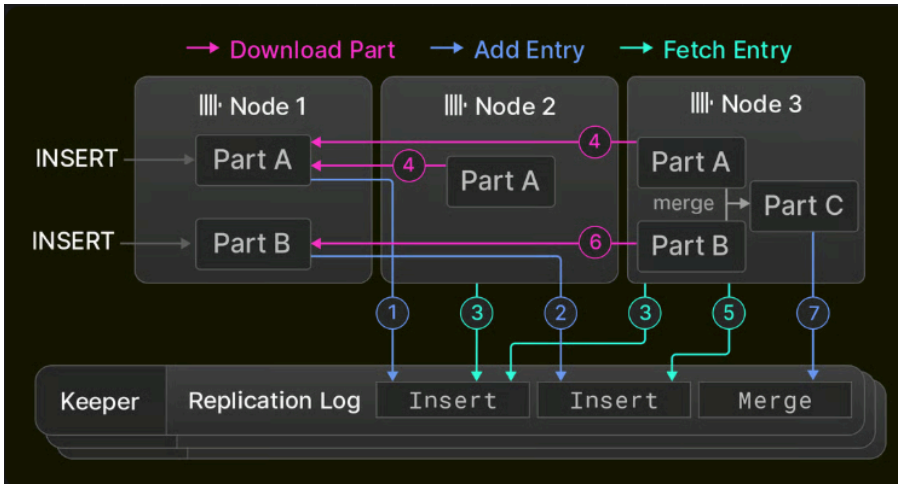



Pertama, storage layer atau biasa disebut MergeTree family. Di layer ini setiap INSERT menjadi satu part, yaitu folder yang berisi satu file per kolom, file .mrk, dan primary.idx. Baris dalam part terurut menurut ORDER BY, lalu dibagi lagi menjadi kelompok-kelompok yang berisi 8.192 baris berurutan. Satu kelompok ini disebut granule yaitu unit terkecil yang dibaca

clickhouse saat query dijalanka. Di arsitektur ini setiap granule hanya punya satu entri di primary index atau sparse, jadi walaupun tabelnya besar index tetap tersimpan di memory. Part yang tebentuk dari insert ini bersifat immutable, lalu proses merge di background menggabungkan part kecil tersebut menjadi part besar.




Kedua, query processing layer atau bagian yang menjalankan query. Cara jalannya sendiri yaitu query diproses sesuai urutan oleh parser, planner, dan physical plan builder, lalu plan executor menjalankan rencana tersebut. Pada saat menjalankan, data tidak diproses satu baris demi satu baris, tetapi per potongan, disebut chunk, yang berisi banyak baris atau biasa disebut vectorized. Pada gambar, paralelisasi terjadi di tiga tingkat dimana beberapa nilai sekaligus oleh SIMD unit dalam satu core, beberapa chunk oleh banyak core dalam satu node, dan beberapa shard oleh banyak node. Saat membaca tabel MergeTree, primary index digunakan untuk melewati granule yang tidak memenuhi filter, dan hanya kolom yang disebut di query yang dibaca. Karena itu biaya agregasi mengikuti jumlah byte kolom yang dipakai, bukan dari lebar baris.




Ketiga, distributed data processing, yang masih bagian dari storage layer dan dipakai saat ClickHouse berjalan di banyak node atau biasa disebut cluster. Di cluster, data satu tabel bisa dipecah ke beberapa node, dan potongan tabel itu disebut shard. Supaya data tidak hilang kalau satu node mati, setiap shard juga bisa replication ke node lain, memakai ReplicatedMergeTree. Untuk membuat semua salinan tetap sama, ada ClickHouse Keeper yang bertugas sebagai penjaga. Dimana Keeper ini kompatibel dengan ZooKeeper dan memakai Raft untuk mencapai kesepakatan antar node yang ada. Selanjutnya, untuk membaca dari beberapa shard sekaligus, digunakan tabel Distributed yang menyebarkan query ke semua shard lalu menggabungkan hasilnya. Replication ini bersifat asinkron secara default, sehingga sebuah node bisa saja membaca data yang tidak terbarukan.

**Metodologi Pemodelan Data**

Pemodelan data pada ClickHouse bukan dimulai dari normalisasi seperti pada relational model, tetapi berangkat dari query. Langkahnya terdiri dari,

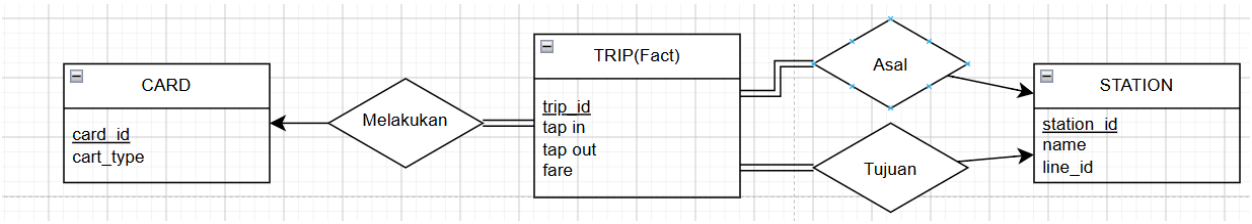
1. Menulis daftar query analitik beserta filter dan GROUP BY-nya.
2. Menetapkan grain tabel fakta, yaitu apa yang diwakili satu baris di tabel.
3. Denormalisasi, dimana kita menggabungkan dimensi kecil ke tabel fakta satu kali saat ETL, jadi query tidak perlu join.
4. Memilih ORDER BY dari kolom yang paling sering difilter. Ini sesuai panduan ClickHouse yang mengutamakan kolom di clause WHERE dan kolom yang membuat data saling berdekatan, sehingga lebih mudah dikompresi. PARTITION BY dipakai supaya rentang waktu yang tidak diminta bisa dilewati, atau biasa disebut partition pruning.
5. Memilih tipe data sekecil mungkin, misalnya UInt16 untuk angka kecil dan LowCardinality untuk teks yang berulang.

Hasil pemodelan data berbeda untuk tiap DBMS. Relasional menormalkan data menjadi beberapa tabel dengan foreign key, sedangkan ClickHouse meratakannya menjadi satu fact tabel lebar atau biasa disebut wide table.

# BAB II
# Studi Kasus

**Analitik Tap-in/Tap-out "TransitTap"**

Dalam kasus ini saya memakai TransitTap, yaitu operator transportasi umum buatan yang mencatat tiap perjalanan saat penumpang tap-in dan tap-out di stasiun, dimana tim analitik dapat membuat laporan dari seluruh riwayat tersebut. Kasus ini dipilih karena datanya dapat besar, terus bertambah, dan jarang diubah, sesuai penggunaan utama ClickHouse. Karena tidak memakai data real life, datanya saya generate dengan script Python dan seed tetap, supaya hasilnya bisa diulang. Jumlahnya saya tentukan sendiri, yaitu 120 stasiun (8 koridor), 200.000 kartu terbagi menjadi regular 70%, student 22% dan senior 8%, dan 5.000.000 perjalanan pada 1 Januari-30 Juni 2025, dengan jam sibuk di pagi dan sore serta akhir pekan yang lebih sepi. Skala yang dipilih cukup besar tetapi tetap cukup di perangkat untuk melakukan studi kasus ini. Mulai dari Stasiun asal, stasiun tujuan, dan kartu dipilih random, sehingga hasilnya dipakai untuk membandingkan DBMS, bukan untuk menyimpulkan kebiasaan penumpang. Kedua DBMS meload file CSV yang sama. Model ER dan kebutuhan ada di bawah ini.




| Id | Kebutuhan | Sifat beban |
|---|---|---|
| Q1 | Perjalanan dan pendapatan per hari | full scan, 2 kolom |
| Q2 | 10 stasiun asal tersibuk bulan Maret | range filter waktu (±17% data) |
| Q3 | Jumlah tap per jam per koridor | full scan |
| Q4 | Pendapatan per tipe kartu per bulan | full scan + data kartu |
| Q5 | Kartu aktif unik per minggu | COUNT DISTINCT |
| Q6 | 10 perjalanan terakhir satu kartu | point lookup |
| W | Ubah tarif satu perjalanan | update satu baris |

**Model data per DBMS**

Model data di tiap DBMS dibawah ini diturunkan dari model ER yang sama. Di ClickHouse datanya disimpan di satu wide table hasil denormalisasi, yang diisi dari tabel

mentah lewat satu INSERT la.u SELECT dengan JOIN ke station dan card. ORDER BY-nya origin_id lalu tap_in, karena dua kolom itu yang paling sering difilter berdasarkan query.

```
CREATE TABLE trips (
  trip_id UInt64, card_id UInt32, card_type LowCardinality(String),
  origin_id UInt16, origin_name LowCardinality(String), line_id
UInt8, dest_id UInt16,
  tap_in DateTime, tap_out DateTime, fare UInt32
) ENGINE = MergeTree PARTITION BY toYYYYMM(tap_in) ORDER BY
(origin_id, tap_in);
```

Di PostgreSQL datanya dinormalisasi menjadi tiga tabel, yaitu

1. stations(station_id PK, name, line_id)
2. cards(card_id PK, card_type)
3. trips(trip_id PK, card_id FK, origin_id FK, dest_id FK, tap_in, tap_out, fare).

Selain itu dibuat B-tree pada (tap_in) dan (card_id, tap_in), supaya query berdasarkan waktu dan berdasarkan card bisa memakai index.

**Contoh Query**

Query ini adalah Q2 di table kebutuhan analitik untuk kasus ini

| PostgreSQL | Clickhouse |
|---|---|
| `SELECT s.name, count(*) AS taps FROM trips t JOIN stations s ON s.station_id = t.origin_id WHERE t.tap_in >= '2025-03-01' AND t.tap_in < '2025-04-01' GROUP BY s.name ORDER BY taps DESC LIMIT 10;` | `SELECT origin_name AS name, count() AS taps FROM trips WHERE tap_in >= '2025-03-01' AND tap_in < '2025-04-01' GROUP BY name ORDER BY taps DESC LIMIT 10;` |

Dapat dilihat query untuk kedua DBMS mirip tapi ada perbedaan dimana Clickhouse tidak memerlukan join karena nama Station sendiri sudah ada di tablenya Query. DDL dari kedua ini ada di repository kode ini.

Analisis Perbandingan Kerja

DBMS yang digunakan untuk membandingkan kinerjanya dengan Clickhouse adalah PostgreSQL 16 sedangkan Clickhouse sendiri itu di versi 25.8.
Implementasi dijalankan di laptop dengan Containerization tiap DBMS dengan batas 4 CPU dan 4 GB dimana hanya aktif satu setiap waktu. Untuk Clickhouse di implementasi ini tidak dimanfaatkan replication dan sharding dari Clickhouse, jadi hanya storage dan query processing layer yang dipakai.

| Komponen | Spesifikasi |
|---|---|
| OS | Windows 11 |
| Prosesor | Intel Core i5-12500H |
| Memory | 16 GB |
| Docker | Docker 29.4.3, Compose v5.1.3 (Docker Desktop) |
| PostgreSQL | 16, image postgres:16 |
| ClickHouse | 25.8, image clickhouse/clickhouse-server:25.8 |
| Client | Python 3.12 (image python:3.12-slim) dengan psycopg, clickhouse-connect. |

Data Uji

Untuk data uji yang digunakan di generate oleh python script yang sama untuk kedua DBMS. Keduanya membuat file CSV yang sama tetapi di Clickhouse tabel utamanya hanya trips, karenda data station dan card included didalamnya.

| Data | Jumlah |
|---|---|
| stations | 120 |
| cards | 200 |
| trips | 5.000.000 |
| Total | 5.200.120 |

**Metode Pengujian**

Pada metode pengujian ini tiap query dijalankan sekali sebagai testing awal, lalu diukur 5 kali, dan yang dilaporkan adalah median dari kelima hasil itu. Seluruh data diload di RAM, jadi yang diukur adalah warm cache, yaitu kondisi saat data sudah ada di memory. Kondisi sebaliknya disebut cold start, yaitu saat data masih harus dibaca dari disk.

Cara yang dipakai ini mengacu ke cara kerja ClickBench yang merupakan cara benchmark publik yang digunakan ClickHouse. Di ClickBench tiap query dijalankan tiga kali, dan run pertama dianggap cold karena cache OS dibersihkan dan database di restart lebih dulu. Dua run berikutnya dianggap hot dan diambil yang tercepat. ClickBench juga mematikan result cache, tetapi cache data sumber boleh dipakai. Dua aturan terakhir itu diterapkan di sini, hanya saja cache tidak dibersihkan sebelum run pertama dan memakai median dari lima run. Karena itu angka di laporan ini setara dengan run hot di ClickBench, bukan run cold.

Sebelum waktunya dibandingkan, hasil tiap query dicek dulu supaya sama di kedua DBMS, yaitu jumlah baris dan total angkanya. Hasilnya semua sama, jadi yang dibandingkan adalah test yang sama.

Untuk PostgreSQL, beberapa pengaturan diubah dadulu supaya perbandingannya menjadi adil. Memory untuk cache data dinaikkan menjadi 1 GB, memory untuk satu operasi seperti sort menjadi 128 MB, dan shared memory container diberi 1 GB. Selain itu parallel query dibuat lebih mudah dipilih, yaitu dengan mengubah parallel_setup_cost dan parallel_tuple_cost menjadi 0, dan JIT dimatikan. Cara ini saya dapatkan dari berdiskusi dengan AI. Dimana alasannya, menurut dokumentasi PostgreSQL paralelisme hanya dipakai kalau query planner menilai itu paling cepat berdasarkan biaya, dan pada pengaturan bawaan beberapa query di data ini ternyata tidak mendapat paralel plan. Sementara itu ClickHouse memakai pengaturan bawaan.

**Hasil Pengujian**

Median waktu tiap query. Hasil ini berasal dari script yang menjalankan tiap query sekali sebagai warm-up lalu 5 kali, dan menyimpan mediannya ke file json.

| Query | PostgreSQL | ClickHouse | CH vs PG |
|---|---|---|---|
| Q1 Trip dan pendapatan per hari | 910 ms | 32 ms | CH 28× |
| Q2 10 station tersibuk (Maret) | 83 ms | 20 ms | CH 4,1× |
| Q3 Tap per jam per koridor | 2,12 s | 23 ms | CH 93× |
| Q4 Pendapatan per tipe card per bulan | 1,92 s | 136 ms | CH 14× |
| Q5 Card aktif unik per minggu | 2,10 s | 145 ms | CH 14× |
| Q6 10 trip terakhir satu card | 2,6 ms | 14 ms | PG 5,4× |
| W Update 1 baris (by id) | 2,7 ms | 73 ms | PG 27× |

**Analisis Hasil Pengujian**

Dari hasil pengujian tidak ada satu DBMS yang unggul di semua jenis operasi. Untuk perbedaan kinerjanya sendiri yang paling menentukan adalah jenis querynya, yaitu apakah query itu membaca hampir semua data atau hanya mengambil sedikit data. Dari situ hasilnya dibagi menjadi tiga kelompok.

Pertama, query yang membaca hampir semua baris untuk dihitung, yaitu Q1, Q3, Q4, dan Q5. Di kelompok ini ClickHouse paling cepat, yaitu 28 kali lebih cepat dari PostgreSQL pada Q1 dan 93 kali pada Q3. Penyebabnya, ClickHouse menyimpan data per kolom, jadi untuk Q1 yang hanya butuh dua kolom, yaitu tap_in dan fare, ia cukup membaca dua kolom itu saja. PostgreSQL menyimpan data per baris, jadi walaupun hanya butuh dua kolom ia tetap membaca seluruh isi tabel. Selain itu pada Q3 dan Q4 PostgreSQL harus menggabungkan

tabel stations dan cards di setiap query, sedangkan di ClickHouse datanya sudah digabung di satu tabel.

Kedua, query yang hanya mengambil sedikit data, yaitu Q2 dan Q6. Di kelompok ini keunggulan ClickHouse mengecil atau malah terbalik. Q2 hanya menyentuh sekitar 17% baris, jadi PostgreSQL terbantu oleh index pada tap_in dan selisihnya dengan ClickHouse hanya 4,1 kali. Pada Q6 yang mencari perjalanan satu card, PostgreSQL (2,6 ms) malah lebih cepat dari ClickHouse (14 ms). Penyebabnya, PostgreSQL punya index untuk card_id, yaitu semacam daftar pencarian, sehingga langsung menemukan baris yang dicari. ClickHouse tidak punya index itu karena tabelnya diurutkan berdasarkan origin_id dan tap_in, jadi ia harus memeriksa seluruh kolom card_id. Kalau filternya searah dengan urutan itu, seperti pada Q2 yang hanya butuh 20 ms, ClickHouse tetap cepat.

Ketiga, update satu baris. Update memakan 2,7 ms di PostgreSQL, tetapi 73 ms di ClickHouse. Penyebabnya, ClickHouse tidak mengubah data di tempat. Perintah updatenya berjalan sebagai mutation, yaitu proses yang menulis ulang kolom yang terkena, jadi lebih berat. Dokumentasi ClickHouse sendiri menyebut operasi ini tidak dirancang untuk sering dipakai. Karena itu mengoreksi data per baris mahal di ClickHouse.

Jadi ClickHouse paling bagus untuk menghitung data yang besar atau OLAP, sedangkan PostgreSQL lebih baik untuk mencari atau mengubah beberapa baris saja alias OLTP. Karena itu pilihan DBMS bergantung pada jenis query yang paling sering dijalankan.

**Storage dan pemodelan**

| DBMS | Ukuran di disk | Rincian | Load time |
|---|---|---|---|
| PostgreSQL | 723 MiB | tabel 365 MiB + index 357 MiB | 99 s |
| ClickHouse | 95 MiB | terkompresi, tanpa index terpisah | 21 s |

Waktu load tidak sepenuhnya setara karena cara loadnya berbeda. Seperti PostgreSQL load dari file CSV lalu membuat dua index dan ClickHouse load tabel mentah lalu menggabungkannya menjadi satu tabel. Waktu hanya diukur sekali, jadi bisa berbeda antar run. Dari sisi ukuran, ClickHouse paling kecil, yaitu 7,6 kali lebih kecil dari PostgreSQL. Penyebabnya, ClickHouse menyimpan data per kolom dan barisnya sudah terurut, jadi nilai yang mirip saling berdekatan dan lebih mudah dikompresi. Selain itu ClickHouse tidak membuat index tambahan untuk tiap baris, sedangkan PostgreSQL membuatnya, sehingga ukurannya bertambah sebesar 357 MiB.

Dari sisi pemodelan, PostgreSQL menyimpan data di beberapa tabel yang saling terhubung, jadi nama station hanya disimpan satu kali dan mengganti nama station cukup dengan satu UPDATE. Datanya juga terjaga konsisten karena PostgreSQL punya aturan yang menghubungkan antar tabel, disebut foreign key. Di ClickHouse nama station disalin ke setiap trip, jadi kalau nama station berubah, semua salinannya harus diubah dan tidak ada aturan yang menjaga konsistensinya. Sebagai gantinya, query analitik atau OLAP cukup membaca satu tabel tanpa join. Di ClickHouse cepat atau lambatnya query juga bergantung pada urutan

tabelnya. Filter yang searah urutan itu, seperti station dan waktu, tetap cepat, sedangkan filter lain seperti card_id pada Q6 jadi lebih lambat.

**Kesimpulan**

Dari hasil pengujian, ClickHouse unggul untuk analitik atas tabel fakta yang besar. Pada Q1 dan Q3 ClickHouse 28 sampai 93 kali lebih cepat dari PostgreSQL, dan datanya jauh lebih kecil. Penyebabnya, ClickHouse menyimpan tiap kolom secara terpisah, jadi ia hanya membaca kolom yang dipakai, lalu mengerjakannya secara paralel.

Di sisi lain, ClickHouse juga punya kelemahan yang terukur. Query yang mencari satu baris lewat kolom di luar ORDER BY (Q6) dan update satu baris dimana lebih lambat. Selain itu ClickHouse tidak punya foreign key, jadi kalau data station berubah, semua salinannya harus diubah.

Karena itu arsitektur yang masuk akal adalah memakai DBMS transaksi seperti PostgreSQL sebagai sumber data utama, lalu mengalirkan datanya berkala ke ClickHouse untuk laporan dan dasbor. Dengan begitu kelemahan masing-masing saling tertutup.

Dari pengujian ini ada empat lesson learned. Hasil pengujian bergantung pada penyetelan awal, jadi PostgreSQL perlu disetel supaya perbandingannya adil. Hasil query di tiap DBMS juga perlu dicocokkan dulu sebelum waktunya dibandingkan. Selain itu urutan tabel di ClickHouse harus diputuskan dari daftar query, karena mengubahnya berarti membangun ulang tabel.

Keterbatasan. Data bersifat sintetis dan dibuat acak, hanya memakai satu node, dan diukur pada kondisi warm cache. PostgreSQL disetel, sedangkan ClickHouse memakai pengaturan bawaan. ClickHouse dalam mode distributed dan skip index juga tidak diuji.

Referensi


1. Wikipedia, "ClickHouse", https://en.wikipedia.org/wiki/ClickHouse.
2. Index Ventures, "ClickHouse: Bringing the Power of Data to Everyone", https://www.indexventures.com/perspectives/clickhouse-bringing-the-power-of-data/.
3. ClickHouse Docs, "Adopters", https://clickhouse.com/docs/about-us/adopters.
4. ClickHouse Docs, "A Practical Introduction to Primary Indexes in ClickHouse", https://clickhouse.com/docs/guides/best-practices/sparse-primary-indexes.
5. ClickHouse Docs, "Choosing a Primary Key", https://clickhouse.com/docs/best-practices/choosing-a-primary-key.
6. ClickHouse, "ClickBench: a Benchmark For Analytical Databases", https://github.com/ClickHouse/ClickBench.
7. ClickHouse Docs, "ALTER TABLE ... UPDATE Statement", https://clickhouse.com/docs/sql-reference/statements/alter/update.
8. PostgreSQL 16 Documentation, "How Parallel Query Works", https://www.postgresql.org/docs/16/how-parallel-query-works.html.
9. Cloudflare Blog, "HTTP Analytics for 6M requests per second using ClickHouse", 2018, https://blog.cloudflare.com/http-analytics-for-6m-requests-per-second-using-clickhouse/.
10. ClickHouse Docs, "Architecture overview" , https://clickhouse.com/docs/academic_overview


Lampiran : Github (https://github.com/FaqihMSY/Clickhouse-vs-PostgreSQL-Study-Case.git)